

SCSR2043-01

OPERATING SYSTEMS

Group 4: I9 CORE

Virtual Memory Management - Replacement Algorithm: FIFO, LRU

Lecturer: Dr Farkhana Binti Muchtar

NAME	MATRIC NO.
KUGAANN SURESH	A24CS0098
KESAVA A/L VIKNESWARAN	A24CS0094
MAHATHEVAN A/L KATHIRAVAN	A24CS0108
MOHAMMED MOHSEN MOHAMMED ALSAKKAF	A23CS4026

TABLE OF CONTENTS

1.0 Title & Abstract	**2**
2.0 Introduction	**3**
3.0 Problem Statement	**4**
4.0 Related Works	**5**
5.0 Methodology	**6**
6.0 Design & Implementation	**7**
7.0 Results & Discussion	**8**
8.0 Future Work & Conclusion	**9**
9.0 Acknowledgement	**10**
10. References	**11**

1.0 Title & Abstract

This project demonstrates the efficiency of Virtual Memory Management specifically focusing on Page Replacement Algorithms. The study compares two fundamental algorithms: First-In-First-Out (FIFO) and Least Recently Used (LRU). To visualize these concepts in a real-world setting, a Web Browser Cache simulation was developed. In this scenario, "web pages" represent memory pages, and the "browser cache limit" represents the number of available frames. The simulation accepts a reference string of visited websites and calculates the number of page faults (cache misses) for both algorithms. The results aim to determine which algorithm minimizes data retrieval latency and optimizes memory usage for a standard user browsing session.

2.0 Introduction

Operating Systems utilize Virtual Memory to execute processes that may exceed the size of the physical memory (RAM). A crucial component of this system is Demand Paging, where data is loaded into memory only when needed. However, physical memory is finite. When the memory is full, the Operating System must decide which existing page to remove (swap out) to make room for a new page. This decision is governed by Page Replacement Algorithms.

In this project, we explore two widely used algorithms: FIFO and LRU. FIFO removes the oldest page in memory regardless of how often it is accessed, while LRU removes the page that has not been used for the longest period. To make these abstract concepts tangible, this project applies them to a Web Browser Cache scenario. Web browsers store recently visited page data in a local cache to load pages faster. Since cache space is limited, the browser must constantly decide which data to keep and which to discard. This project simulates this behavior to evaluate the performance trade-offs between FIFO and LRU.

3.0 Problem Statement

In modern computing, users frequently multi-task, keeping numerous applications and browser tabs open simultaneously. A significant problem arises when the limited physical memory (frames) fills up. An inefficient page replacement strategy can lead to "Thrashing," where the system spends more time swapping data in and out of the disk than executing tasks, causing the computer to freeze or lag.

Specifically in web browsing, if the cache management is poor, the user experiences slow load times because the browser must re-download data that was just deleted. The problem this project addresses is: **How do different replacement algorithms (FIFO vs. LRU) impact the number of page faults (cache misses) in a constrained memory environment?** We aim to identify which algorithm provides a better user experience by minimizing the need to fetch data from the slower secondary storage/network.

4.0 Related Works

The concept of Page Replacement Algorithms has been extensively studied in the field of Operating Systems. According to Silberschatz et al. (2013), virtual memory management relies heavily on the efficiency of these algorithms to minimize the "effective access time" of memory.

4.1 FIFO and Belady's Anomaly

The First-In-First-Out (FIFO) algorithm is the simplest baseline for page replacement. However, it is documented in literature to suffer from **Belady's Anomaly**, a phenomenon where increasing the number of available memory frames results in *more* page faults rather than fewer. This behavior makes FIFO unpredictable for certain reference strings, despite its ease of implementation.

4.2 LRU and Stack Algorithms

To counter the inefficiencies of FIFO, the Least Recently Used (LRU) algorithm is often proposed. Unlike FIFO, LRU is considered a "stack algorithm," meaning it never exhibits Belady's Anomaly. Research by Tanenbaum (2014) highlights that while LRU approximates the optimal replacement policy well, it is computationally expensive to implement in hardware because it requires constant updates to the "time of use" for every single page access.

4.3 Web Caching Variants

In the specific domain of web browsing, variations of these OS algorithms are used. Modern browsers often implement complex hybrid strategies, but the core principles remain rooted in the classical LRU and FIFO logic. This project builds upon these established theoretical frameworks by visualizing them in a simplified browser environment.

5.0 Methodology

To evaluate the performance of page replacement algorithms, a simulation program was developed using the Python programming language. The simulation models a Web Browser Cache environment where "pages" are represented by URL strings (e.g., "google.com", "youtube.com").

5.1 Simulation Setup

The system is designed with a configurable "Cache Size" (representing the number of frames in physical memory). The input consists of a "Reference String," which is a sequence of websites visited by the user.

5.2 Algorithms Implemented

1. **FIFO (First-In-First-Out):** The simulation utilizes a queue data structure. When a user visits a new website and the cache is full, the website that entered the cache earliest is removed to make space.
2. **LRU (Least Recently Used):** The simulation tracks the usage history of every page in the cache. When a replacement is needed, the system identifies and removes the website that has not been accessed for the longest duration.

5.3 Data Collection

The simulation processes the same reference string through both algorithms. For every step, the system records:

- **Hit:** The page was found in the cache (no delay).
- **Fault:** The page was not in the cache (requires loading).

The final output compares the Total Page Faults for both algorithms to determine which is more efficient for the given browsing scenario.

6.0 Design & Implementation

The simulation was implemented using Python (v3.x) within the PyCharm IDE. The design follows an Object-Oriented approach with a class named `BrowserCacheSimulation`.

6.1 Data Structures Used

- **List (Array):** Used to represent the `cache` frames. This allows for index-based operations which are essential for identifying the "First-In" element or the "Least Recently Used" element.
- **Input List:** A predefined list of strings (`website_history`) representing the sequence of user URL requests.

6.2 Algorithm Logic

- **FIFO Logic:** The program checks if the requested page is in the list. If not and the list is full, the element at `index 0` (the head) is popped, and the new page is appended to the end.
- **LRU Logic:** The program utilizes Python's list manipulation features. When a page is accessed (Hit), it is removed from its current position and re-appended to the end of the list. This ensures that `index 0` always contains the page that was accessed longest ago.

6.3 User Interface

The application runs in a Command Line Interface (CLI). It outputs a formatted table showing the Step Number, URL Request, Action (Hit/Fault), and the current Cache State for every step of the simulation.

THE CODE :

```python
import time

class BrowserCacheSimulation:
    def __init__(self, capacity):
        self.capacity = capacity  # Number of frames (Cache Size)

    def run_fifo(self, pages):
        print(f"\n--- Simulation: FIFO Algorithm (Cache Size: {self.capacity})
---")
        print(f"{'Step':<5} | {'URL Request':<15} | {'Action':<10} | {'Cache
State'}")
        print("-" * 60)

        cache = []
        page_faults = 0

        for i, page in enumerate(pages):
            action = "Hit"
            if page not in cache:
                action = "FAULT"
                page_faults += 1
                if len(cache) < self.capacity:
                    cache.append(page)
                else:
                    cache.pop(0)  # Remove the first item (FIFO)
                    cache.append(page)

            # Print status
            print(f"{i+1:<5} | {page:<15} | {action:<10} | {cache}")
            # time.sleep(0.5) # Uncomment this if you want to see it animate
slowly in terminal

        print("-" * 60)
        print(f"Total FIFO Page Faults: {page_faults}\n")
        return page_faults

    def run_lru(self, pages):
        print(f"\n--- Simulation: LRU Algorithm (Cache Size: {self.capacity})
---")
        print(f"{'Step':<5} | {'URL Request':<15} | {'Action':<10} | {'Cache
State'}")
        print("-" * 60)

        cache = []
```

```
        page_faults = 0

        for i, page in enumerate(pages):
            action = "Hit"
            if page not in cache:
                action = "FAULT"
                page_faults += 1
                if len(cache) < self.capacity:
                    cache.append(page)
                else:
                    # Logic for LRU is handled by the order of the list
                    # The first item (index 0) is always the Least Recently
Used
                    cache.pop(0)
                    cache.append(page)
            else:
                # If page is in cache, move it to the end (Most Recently Used)
                cache.remove(page)
                cache.append(page)

            # Print status
            print(f"{i+1:<5} | {page:<15} | {action:<10} | {cache}")
            # time.sleep(0.5)

        print("-" * 60)
        print(f"Total LRU Page Faults: {page_faults}\n")
        return page_faults

# --- Real World Scenario Inputs ---
# Scenario: A student researching for an assignment
# Cache Size = 3 tabs
simulation = BrowserCacheSimulation(capacity=3)

# The sequence of websites visited
website_history = [
    "google.com", "utm.my", "chatgpt.com", "google.com",
    "wikipedia.org", "utm.my", "youtube.com", "wikipedia.org",
    "google.com", "utm.my", "chatgpt.com", "wikipedia.org"
]

print("STARTING BROWSER CACHE SIMULATION...")
print(f"Scenario: Student Researching | Open Tabs Allowed:
{simulation.capacity}")
print(f"Websites Visited: {website_history}")

# Run both algorithms
fifo_faults = simulation.run_fifo(website_history)
```

```
lru_faults = simulation.run_lru(website_history)

# Final Summary
print("=== FINAL RESULTS ===")
print(f"FIFO Total Faults: {fifo_faults}")
print(f"LRU Total Faults:  {lru_faults}")
if lru_faults < fifo_faults:
    print("Conclusion: LRU was more efficient.")
elif fifo_faults < lru_faults:
    print("Conclusion: FIFO was more efficient.")
else:
    print("Conclusion: Both performed the same.")
```

```
C:\Users\youca\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\youca\PyCharmMiscProject\browser_cache_sim.py
STARTING BROWSER CACHE SIMULATION...
Scenario: Student Researching | Open Tabs Allowed: 3
Websites Visited: ['google.com', 'utm.my', 'chatgpt.com', 'google.com', 'wikipedia.org', 'utm.my', 'youtube.com', 'wikipedia.org', 'google.com', 'utm.my', 'chatgpt.com', 'wikipedia.org']

--- Simulation: FIFO Algorithm (Cache Size: 3) ---
Step  | URL Request     | Action     | Cache State
------------------------------------------------------------
1     | google.com      | FAULT      | ['google.com']
2     | utm.my          | FAULT      | ['google.com', 'utm.my']
3     | chatgpt.com     | FAULT      | ['google.com', 'utm.my', 'chatgpt.com']
4     | google.com      | Hit        | ['google.com', 'utm.my', 'chatgpt.com']
5     | wikipedia.org   | FAULT      | ['utm.my', 'chatgpt.com', 'wikipedia.org']
6     | utm.my          | Hit        | ['utm.my', 'chatgpt.com', 'wikipedia.org']
7     | youtube.com     | FAULT      | ['chatgpt.com', 'wikipedia.org', 'youtube.com']
8     | wikipedia.org   | Hit        | ['chatgpt.com', 'wikipedia.org', 'youtube.com']
9     | google.com      | FAULT      | ['wikipedia.org', 'youtube.com', 'google.com']
10    | utm.my          | FAULT      | ['youtube.com', 'google.com', 'utm.my']
11    | chatgpt.com     | FAULT      | ['google.com', 'utm.my', 'chatgpt.com']
12    | wikipedia.org   | FAULT      | ['utm.my', 'chatgpt.com', 'wikipedia.org']
------------------------------------------------------------
Total FIFO Page Faults: 9


--- Simulation: LRU Algorithm (Cache Size: 3) ---
Step  | URL Request     | Action     | Cache State
------------------------------------------------------------
1     | google.com      | FAULT      | ['google.com']
2     | utm.my          | FAULT      | ['google.com', 'utm.my']
3     | chatgpt.com     | FAULT      | ['google.com', 'utm.my', 'chatgpt.com']
4     | google.com      | Hit        | ['utm.my', 'chatgpt.com', 'google.com']
5     | wikipedia.org   | FAULT      | ['chatgpt.com', 'google.com', 'wikipedia.org']
6     | utm.my          | FAULT      | ['google.com', 'wikipedia.org', 'utm.my']
7     | youtube.com     | FAULT      | ['wikipedia.org', 'utm.my', 'youtube.com']
8     | wikipedia.org   | Hit        | ['utm.my', 'youtube.com', 'wikipedia.org']
9     | google.com      | FAULT      | ['youtube.com', 'wikipedia.org', 'google.com']
10    | utm.my          | FAULT      | ['wikipedia.org', 'google.com', 'utm.my']
11    | chatgpt.com     | FAULT      | ['google.com', 'utm.my', 'chatgpt.com']
12    | wikipedia.org   | FAULT      | ['utm.my', 'chatgpt.com', 'wikipedia.org']
------------------------------------------------------------
Total LRU Page Faults: 10

=== FINAL RESULTS ===
FIFO Total Faults: 9
LRU Total Faults:  10
Conclusion: FIFO was more efficient.

Process finished with exit code 0
```

7.0 Results & Discussion

The simulation was executed with a **Cache Size of 3** (simulating a browser limited to 3 active tabs in memory) and a reference string of 12 website visits.

7.1 Quantitative Results

- **Total Requests:** 12
- **FIFO Total Page Faults:** 9
- **LRU Total Page Faults:** 10

7.2 Analysis

In this specific "Student Researching" scenario, the FIFO algorithm performed slightly better than LRU, resulting in one fewer page fault.

- **FIFO Performance:** FIFO benefited from the cyclic nature of the specific input string. The pages that were evicted were often not immediately needed again until after the cache had cycled, minimizing the "Belady's Anomaly" effect in this instance.
- **LRU Performance:** LRU incurred a higher fault rate because the user pattern frequently revisited pages just after they had drifted to the "least recently used" position (index 0) and were evicted.

7.3 Comparison

While theoretical studies suggest LRU is generally superior for random workloads, this experiment demonstrates that for specific browsing patterns—particularly where a user cycles through a fixed set of websites (e.g., Google -> UTM -> ChatGPT -> Google)—FIFO can occasionally offer equal or better performance with lower computational overhead.

8.0 Future Work & Conclusion

8.1 Conclusion

This project successfully visualized the complex mechanism of Virtual Memory Management through a relatable Web Browser Cache simulation. By implementing the First-In-First-Out (FIFO) and Least Recently Used (LRU) algorithms, we observed how the Operating System manages limited memory frames. Our experimental results showed that while LRU is often theoretically superior, simple algorithms like FIFO can perform efficiently in specific, cyclical browsing patterns.

8.2 Future Work

To further enhance this project, future iterations could include:

1. **Graphical User Interface (GUI):** Replacing the text-based output with a visual animation using libraries like `Tkinter` or `PyGame` to show tabs moving in and out of the stack in real-time.
2. **Dynamic Input:** Allowing the user to input their own list of websites and custom cache sizes during runtime.
3. **Optimal Algorithm Comparison:** Implementing the "Optimal Page Replacement" algorithm (which requires future knowledge) to serve as a baseline for measuring the absolute efficiency of FIFO and LRU.

9.0 Acknowledgement

We would like to express our gratitude to our lecturer, **Dr. Farkhana Binti Muchtar**, for her guidance and support throughout this course. Her insights into Operating System concepts provided the necessary foundation for this project. We also thank Universiti Teknologi Malaysia (UTM) for providing the resources required to complete this simulation.

10. References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). Hoboken, NJ: John Wiley & Sons.

[2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.

[3] Python Software Foundation. (2024). *Python 3.12 Documentation*. Retrieved from https://docs.python.org/3/